Programming is Hard!

- Programming is about convincing machines to do what we want them to do.
- Systematic and rigorous (“No more, no less”).
- The way in which we communicate with machines shapes what we can (or can’t) do.
- …and how easy it is achieve it.

Programming is Hard!

- Programming languages and their features shape what our programs can and cannot do.
- “If the only tool you have is a hammer, you tend to see every problem as a nail.”
- Lets think a bit about what kind of “hammers” we have…

What about PLs?

- Two axes:
  - What kind of runtime errors are allowed?
  - What idioms and paradigms are facilitated by language features?

Things go Wrong

- Different languages allow programs to go wrong in different ways.
- We distinguish between runtime and compile-time errors:
  - Anything goes (e.g. Python)
  - Memory violation errors (e.g. C/C++)
  - Null pointer dereferencing (e.g. Java)
  - Memory safety (e.g. Rust)

Languages as Tools

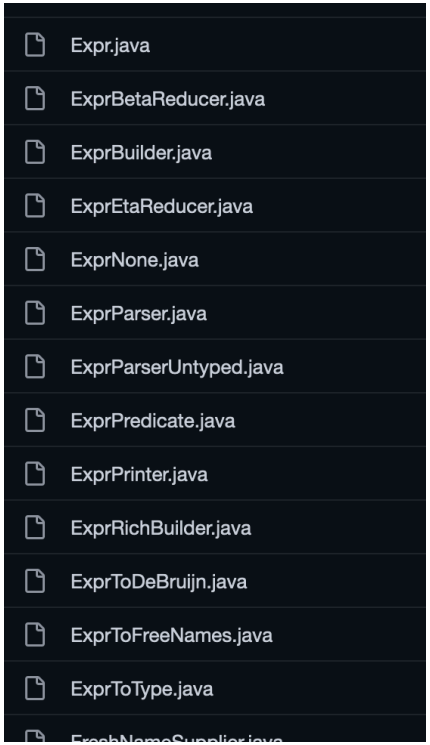
- Beyond errors, languages also provide abstractions that can be better suited for certain problems.
- Languages and their implementations have a very wide range of focus.
- Different problems are better solved using specific (language) features.

Languages as Tools

Writing a compiler / interpreter / static analyzer?

```haskell
data Type
    = Base
    | Arrow Type Type
    deriving (Eq, Ord, Read, Show)

data Term
    = Const
    | Var Int -- deBruijn indexing; the nearest enclosing lambda binds Var 0
    | Lam Type Term
    | App Term Term
    deriving (Eq, Ord, Read, Show)

check :: [Type] -> Term -> Maybe Type
check env Const = return Base
check env (Var v) = atMay env v
check env (Lam ty tm) = Arrow ty <$> check (ty:env) tm
check env (App tm tm') = do
    Arrow i o <- check env tm
    i' <- check env tm'
    guard (i == i')
    return o

eval :: Term -> Term
eval (App tm tm') = case eval tm of
    Lam _ body -> eval (subst 0 tm' body)
eval v = v

subst :: Int -> Term -> Term -> Term
subst n tm Const = Const
subst n tm (Var m) = case compare m n of
    LT -> Var m
```

VS



Languages as Tools

Web development?

```html
<html>
  <head>
    <div>
          <div>

              <form method="post" action="#" id="formvalue" onkeyup="
              drawChart()" />

              </form>
          </div>
    </div>



    <script type="text/javascript" src="https://www.google.com/jsapi"></
      script>
    <script type="text/javascript">

    var bid = 43;
    var ask = 21;


      google.load("visualization", "1", {packages:["corechart"]});
      google.setOnLoadCallback(drawChart);
      function drawChart() {
        var data = google.visualization.arrayToDataTable([
          ['Price', 'Quantity'],
          ['Value #1', bid],
          ['Value #2', ask],
        ]);
```

Javascript

```typescript
type filterFn = (item: string) => bool;

// The higher-order-function takes an array and a function as arguments
function filterItems(arr: string[], fn: filterFn): string[] {
    const newArray: string[] = [];
    arr.forEach(item => {
        if(fn(item)){
          newArray.push(item);
        }
    });
    return newArray;
}

function checkNameLength(name: string) {
  return name.length >= 10;
}

const doctorList = ["DoctorOne", "DoctorTwo", "DoctorThree", "DoctorFour"];

// We are passing the array and a function as arguments to filterItems method.
const output = filterItems(doctorList, checkNameLength);

console.log(output); // ["DoctorThree", "DoctorFour"]
```

Typescript

```elm
view : Model -> Html Msg
view model =
    div
        [ class "todomvc-wrapper"
        , style [ ( "visibility", "hidden" ) ]
        ]
        [ section
            [ class "todoapp" ]
            [ lazy viewInput model.field
            , lazy2 viewEntries model.visibility model.entries
            , lazy2 viewControls model.visibility model.entries
            ]
        , infoFooter
        ]


viewInput : String -> Html Msg
viewInput task =
    header
        [ class "header" ]
        [ h1 [] [ text "todos" ]
        , input
            [ class "new-todo"
            , placeholder "What needs to be done?"
            , autofocus True
            , value task
            , name "newTodo"
            , onInput UpdateField
            , onEnter Add
            ]
            []
        ]
```

Elm

Languages as Tools

Device driver?

```c
#include <linux/module.h>
#include <linux/string.h>
#include <linux/fs.h>
#include <asm/uaccess.h>

// module attributes
MODULE_LICENSE("GPL"); // this avoids kernel taint warning
MODULE_DESCRIPTION("Device Driver Demo");
MODULE_AUTHOR("Appu Sajeev");

static char msg[100]={0};
static short readPos=0;
static int times = 0;

// protoypes,else the structure initialization that follows fail
static int dev_open(struct inode *, struct file *);
static int dev_rls(struct inode *, struct file *);
static ssize_t dev_read(struct file *, char *, size_t, loff_t *);
static ssize_t dev_write(struct file *, const char *, size_t, loff_t *);

// structure containing callbacks
static struct file_operations fops =
{
        .read = dev_read, // address of dev_read
        .open = dev_open,  // address of dev_open
        .write = dev_write, // address of dev_write
        .release = dev_rls, // address of dev_rls
};


// called when module is loaded, similar to main()
int init_module(void)
{
        int t = register_chrdev(89,"myDev",&fops); //register driver with major:89

        if (t<0) printk(KERN_ALERT "Device registration failed..\n");
        else printk(KERN_ALERT "Device registered...\n");

        return t;
}


// called when module is unloaded, similar to destructor in OOP
void cleanup_module(void)
{
```

C/C++

```rust
/// Communicating to the sensor, updates the latest wrench.
/// # Returns
/// `Ok(wrench)` if succeeds, `Err(reason)` if failed.
pub fn update(&mut self) -> Result<Wrench, Error> {
    let res = receive_message(&mut self.port);

    // Regardless of success or failure of receive_message(), request the next single data.
    // If we do not so, after updating failed once, updating will fail everytime due to no reception from the sensor.
    self.request_next_wrench()?;

    let res = match res {
        Ok(res) => res,
        Err(e) => {
            return Err(e);
        }
    };

    let (fx, fy, fz, mx, my, mz) = (0..6)
        .map(|i| 4 + i * 2)
        .filter_map(|start| res.get(start..start + 2))
        .map(|res| i16::from_le_bytes(res.try_into().unwrap()))
        .map(|digital| digital as f64)
        .next_tuple()
        .ok_or(Error::ParseData)?;

    let rated_binary = self.product.rated_binary();
    let force = Triplet::new(fx, fy, fz)
        .map_entrywise(self.rated_wrench.force, |left, right| {
            left / rated_binary * right
        });
    let torque = Triplet::new(mx, my, mz)
        .map_entrywise(self.rated_wrench.torque, |left, right| {
            left / rated_binary * right
        });

    self.last_raw_wrench = Wrench::new(force, torque);

    Ok(self.last_wrench())
}
```

Rust

Languages as Tools

Operating System?

```rust
/// An ELF executable
pub struct Elf<'a> {
    pub data: &'a [u8],
    header: &'a header::Header
}

impl<'a> Elf<'a> {
    /// Create a ELF executable from data
    pub fn from(data: &'a [u8]) -> Result<Elf<'a>, String> {
        if data.len() < header::SIZEOF_EHDR {
            Err(format!("Elf: Not enough data: {} < {}", data.len(), header::SIZEOF_EHDR))
        } else if &data[..header::SELFMAG] != header::ELFMAG {
            Err(format!("Elf: Invalid magic: {:?} != {:?}", &data[..header::SELFMAG], header::ELFMAG))
        } else if data.get(header::EI_CLASS) != Some(&header::ELFCLASS) {
            Err(format!("Elf: Invalid architecture: {:?} != {:?}", data.get(header::EI_CLASS), header::ELFCLASS))
        } else {
            Ok(Elf {
                data,
                header: unsafe { &*(data.as_ptr() as usize as *const header::Header) }
            })
        }
    }

    pub fn sections(&'a self) -> ElfSections<'a> {
        ElfSections {
            data: self.data,
            header: self.header,
            i: 0
        }
    }

    pub fn segments(&'a self) -> ElfSegments<'a> {
        ElfSegments {
            data: self.data,
            header: self.header,
            i: 0
        }
    }
```

Rust

```ocaml
module Rx(Time:Mirage_time.S)(ACK: Ack.M) = struct
  open Tcp_packet
  module StateTick = State.Make(Time)

  (* Individual received TCP segment
     TODO: this will change when IP fragments work *)
  type segment = { header: Tcp_packet.t; payload: Cstruct.t }

  let pp_segment fmt {header; payload} =
    Format.fprintf fmt
      "RX seg seq=%a acknum=%a ack=%b rst=%b syn=%b fin=%b win=%d len=%d"
      Sequence.pp header.sequence Sequence.pp header.ack_number
      header.ack header.rst header.syn header.fin
      header.window (Cstruct.length payload)

  let len seg =
    Sequence.of_int ((Cstruct.length seg.payload) +
    (if seg.header.fin then 1 else 0) +
    (if seg.header.syn then 1 else 0))

  (* Set of segments, ordered by sequence number *)
  module S = Set.Make(struct
      type t = segment
      let compare a b = (Sequence.compare a.header.sequence b.header.sequence)
    end)

  type t = {
    mutable segs: S.t;
    rx_data: (Cstruct.t list option * Sequence.t option) Lwt_mvar.t; (* User receive channel *)
    ack: ACK.t;
    tx_ack: (Sequence.t * int) Lwt_mvar.t; (* Acks of our transmitted segs *)
    wnd: Window.t;
    state: State.t;
  }
```

OCaml

Languages as Tools

- Different lang. impl. provide different trade-offs:
  - Efficiency
  - Correctness
  - Rapid prototyping
  - Ease of refactoring
  - Interoperability

Languages as Tools

Correctness is mandatory, but how hard it is to get it will often depend on the tool / language.

```
yuvraj@DeathNote:~/head-first-c-exercises$ ./find
Search Results:
-----------------------------------
William: SBM GSOH likes sports,TV,dining
Josh:SJM likes sports,movie theatre
Segmentation fault (core dumped)
yuvraj@DeathNote:~/head-first-c-exercises$
```